

AUC Campus Navigator

CSCE 2211 Term Project Report - Fall 2025

The American University in Cairo

Abstract

The AUC Campus Navigator is a graph-based routing system designed to help students, visitors, and faculty efficiently navigate the American University in Cairo campus. By modeling the campus as a network of interconnected nodes and applying shortest-path algorithms, the system provides clear instructions from any starting point to a desired destination. This project aims to reduce confusion—particularly for newcomers—by offering a digital and user-friendly navigation tool accessible through a command-line interface, along with a GUI-based mobile application.

Keywords

Graphs; AUC Campus; Map Navigator; Dijkstra; Shortest Path

1 Introduction

The AUC campus is known for its architectural complexity, open spaces, and diverse walkways connecting academic buildings and student facilities. While this layout offers a rich experience, newcomers often find it difficult to navigate, especially during their first weeks on campus. Physical maps placed throughout campus are sometimes confusing and more helpful to experienced students than to new arrivals.

To support students, visitors, alumni, and newly hired faculty in navigating the campus efficiently, we designed the AUC Campus Navigator. This system applies core data structures and algorithms to deliver accurate routing instructions tailored to the campus layout. It enhances existing navigation tools by offering an interactive interface and a structured pathfinding approach.

2 Problem Definition

Many individuals encounter difficulty identifying the correct walking trajectory between two locations on campus. The non-linear layout, numerous alternative paths, and overlapping naming conventions of classrooms often contribute to confusion. Additionally,

there is currently a lack of a unified digital navigation tool dedicated to the AUC campus environment.

The AUC Campus Navigator addresses these challenges by providing:

- A location search system with ambiguity resolution.
- Computation and display of the shortest route between locations.
- A dynamic interface allowing users to update their current location at any time.

For this phase, the system includes only the main outdoor campus paths. Indoor navigation will be incorporated in future iterations.

3 Methodology

The project models the AUC campus as a weighted graph, where each node represents a campus location and each edge represents a walkable path with an associated distance. The navigation system is composed of the following core components:

- **Graph:** A custom implementation storing all nodes and weighted edges.
- **Min-Heap:** Custom-built to support Dijkstra's priority queue operations.
- **Dijkstra's Algorithm:** Used exclusively for shortest-path computation.
- **Command-Line Interface:** Receives user input, resolves location ambiguity, executes shortest-path queries, and displays step-by-step navigation.
- **GUI Application:** A mobile app integrates the algorithmic functionality to provide interactive navigation with visual components associated with Apple Maps.

4 Specification of Algorithms to be Used

4.1 Graph Construction Pseudocode

`GRAPH-ADD-VERTEX(name):`

```
if name not in hashTable:
    create new VertexNode(name)
    insert into linked list of vertices
    hashTable.insert(name, pointer to vertex)
```

`GRAPH-ADD-EDGE(u, v, w):`

```

ensure u exists using ADD-VERTEX(u)
ensure v exists using ADD-VERTEX(v)
append EdgeNode(v, w) to adjacency list of u
append EdgeNode(u, w) to adjacency list of v    // undirected graph

```

4.2 Hash Table Pseudocode (DJB2 + Linear Probing)

```

HASH(key):
    hash = 5381
    for each character c in key:
        hash = hash * 33 + c
    return hash mod capacity

INSERT(key, value):
    if load factor exceeded:
        rehash()
    index = HASH(key)
    for attempt = 0 to capacity:
        slot = (index + attempt) mod capacity
        if slot empty or deleted:
            place key and value
            return success
        if keys match:
            update value
            return success

```

4.3 Min-Heap Pseudocode

```

INSERT(x):
    place x at end
    while x < parent(x):
        swap x with parent
        move upward

EXTRACT-MIN():
    min = root
    move last element to root
    heapify-down root
    return min

```

```

DECREASE-KEY(i, newValue):
    arr[i] = newValue
    while arr[i] < parent(i):
        swap with parent

```

4.4 Dijkstra's Algorithm Pseudocode

```

DIJKSTRA(Graph, start, destination):
    for each vertex v:
        distance[v] = INF
        previous[v] = NIL
        visited[v] = false

    distance[start] = 0
    heap.insert((start, 0))

    while heap not empty:
        (u, dist) = heap.extractMin()
        if visited[u] continue
        visited[u] = true

        if u == destination break

        for each neighbor v of u:
            newDist = distance[u] + weight(u, v)
            if newDist < distance[v]:
                distance[v] = newDist
                previous[v] = u
                heap.decreaseKey(v, newDist) or heap.insert(v, newDist)

    reconstruct path using previous[]
    return path and total distance

```

5 Data Specifications

The dataset will include:

- A list of campus locations represented as graph nodes.

- Walkable connections between location pairs, represented as weighted edges.
- Estimated walking distances between each pair of connected nodes.

The dataset will be stored in a structured JSON file containing node definitions, and edge lists. Finalization of the dataset is still in progress.

6 Experimental Results

6.1 Test Cases

We plan to evaluate the system using the following scenarios:

- **Test Case 1:** Shortest path from SSE to Library Front Entrance.
- **Test Case 2:** Shortest path from Jameel to Pepsi Entrance.
- **Test Case 3:** Shortest path from Bassily Hall to Watson House.

6.2 Expected Behavior

- Correct shortest path results.
- Accurate nearest facility identification.
- Human-readable directional output.
- Robust matching for ambiguous or partial user input.
- Reasonable time estimates based on edge weights.

7 Analysis and Critique

7.1 Strengths

- Fast and efficient pathfinding using Dijkstra's algorithm.
- User-friendly CLI with intuitive interaction.
- Modular architecture enabling future mobile app development.
- Modern visualization and real-time GPS integration.

7.2 Limitations

- Incomplete dataset in the current phase.
- No indoor navigation.

7.3 Possible Improvements

- Full official mobile application and website with added features.
- Indoor navigation using building floor plans.
- Replace the usage of Apple Maps with our custom map design.
- Accessibility features for users with disabilities.

8 Conclusions

The AUC Campus Navigator demonstrates the effectiveness of graph theory and shortest-path algorithms in solving practical navigation challenges. Through this project, we gained experience in modeling physical environments with data structures, implementing Dijkstra’s algorithm, and designing a user-focused interface. The resulting system provides accurate and efficient navigation across the AUC campus, forming a solid foundation for future enhancements, such as the official website, indoor mapping, and real-time positioning.

Acknowledgements

We would like to thank the AUC Architecture Department for providing access to the campus map. We also acknowledge Eng. Mohamed El-Shazly for providing help in handling the AutoCAD map file and assisting in isolating usable layers. Our teammates who contributed to this project include: Omar Marey, Adham Abotarboush, Ahmed Yamany, and Tarek Ibrahim.

References

1. Dijkstra, E. W. “A Note on Two Problems in Connexion with Graphs.” *Numerische Mathematik*, 1959.
2. AUC Campus Maps and Directions. The Architecture Department - The American University in Cairo.

3. “Graph Algorithms – Dijkstra.” CP-Algorithms.
4. CSCE 2211 Course Notes and Materials.

Appendix: Development Challenges and Solutions

The process of acquiring and preparing the campus dataset presented considerable challenges throughout the project development phase. Our first strategy involved contacting the Architecture Department to obtain an official AutoCAD campus map. However, we encountered significant compatibility and data structure issues when attempting to parse and extract nodes and pathways from the file; the multiple layers, overlapping elements, and formatting inconsistencies made programmatic extraction impractical. We subsequently attempted manual digitisation using Google My Maps to plot nodes and edges, downloading the data as KML and converting it to JSON format. While this approach provided greater control over data accuracy, it proved extremely time-consuming and labour-intensive. Finally, we discovered a Python library that automatically generated all possible nodes and paths based on map dimensions, significantly reducing manual workload. However, this efficiency came with a trade-off: the library produced a substantially larger dataset than anticipated, with numerous auto-generated nodes assigned default labels (e.g., `r1`, `r2`, `...`, `r30`) rather than descriptive location names. Due to the sheer volume of data, completing the manual naming of all nodes proved infeasible within the project scope, resulting in the incomplete dataset acknowledged in the Limitations section.

The implementation of Dijkstra’s algorithm similarly presented unexpected challenges during early development. Our initial attempts to implement the algorithm using various approaches did not produce correct shortest-path results, and we spent considerable time debugging issues stemming from inadequate data organization and inconsistent state management. A significant breakthrough occurred when we recognized the necessity of implementing a custom hash table (using DJB2 hashing with linear probing) and a dedicated data manager class to abstract graph state complexity. These supporting structures fundamentally improved algorithmic correctness by providing efficient node lookup, centralising data organization, and eliminating subtle bugs that plagued earlier implementations. This experience reinforced that algorithm implementation quality depends not merely on correct pseudocode translation, but equally on thoughtful data structure design and system architecture.